

ML CW 2

aw1620, fh720, psb120, sc1320

November 2022

1 Model description and justification

Our team implemented our neural network model using PyTorch. The network comprises of alternating linear and ReLU layers. Since the problem domain is a regression task, the final layer is a linear layer.

Optimization is performed using mini-batching, as this solution provides good trade-off between speed and accuracy when compared to either stochastic or batch gradient descent. The Adam Optimization Algorithm, an extended version of stochastic gradient descent, is used for optimization. It is computationally efficient and well-suited for noisy gradients. Furthermore, its hyper-parameters typically require little tuning.

Input data is standardized, i.e. the data points for each feature are transformed so that their mean is 0 and variance 1. This makes weight updates proportional to the input and improves model accuracy.

2 Evaluation Setup

The accuracy of the model is evaluated using the loss value. It is calculated on the training, evaluation and test data sets. The mean squared error (MSE) loss function is used since the model is used for a regression task. MSE has been chosen instead of mean absolute error to ensure a decent fit to all data points. The difference between loss on the training and test data sets relative to the loss on the training set is used to detect if the model is overfitting.

3 Hyper-Parameter Search and Findings

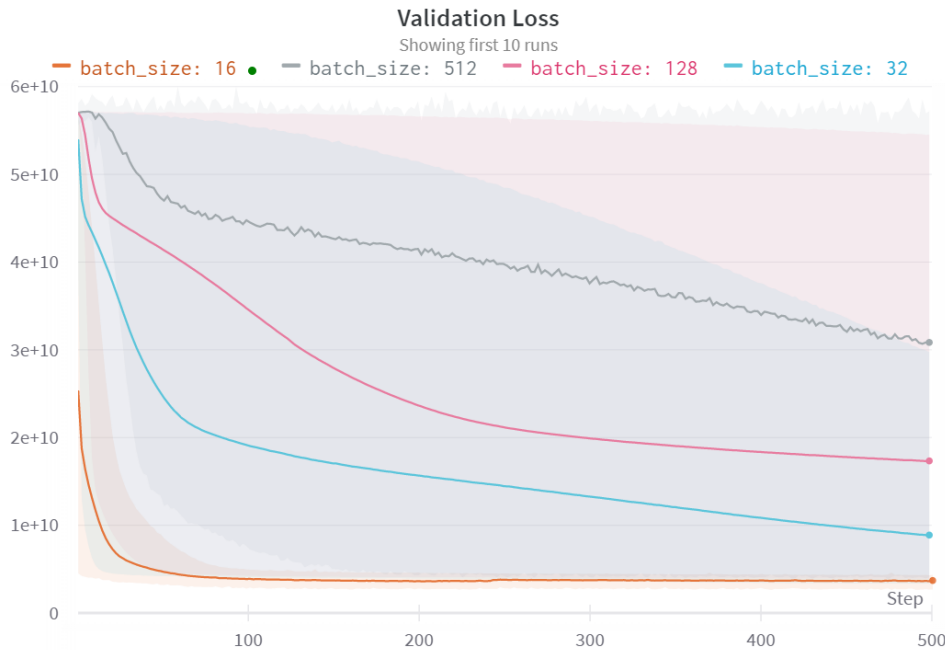
During model training hyper-parameter tuning is used to improve overall speed and performance. The investigated variables are: learning rate, number of epochs, batch size and shape (number of layers and number of neurons per layer). Increasing the learning rate and decreasing the batch size and the number of epochs can improve model speed without sacrificing accuracy. Choosing the right shape of the network influences the balance between bias and variance, and model speed. The model with the lowest loss and highest speed is chosen. We used Weights and Biases track our metrics during hyper-parameter tuning and generate graphs for this report. The below graphs display the mean loss of experiments by the corresponding metric, to visually understand how the parameter affects loss and speed. The background shows max. and min. curves for the corresponding metric.

3.1 Findings

The table below illustrates the results of hyper-parameter tuning. The column 'rel diff' stands for relative difference and represents the difference between test loss and training loss divided by training loss. A large absolute value of 'rel diff' indicates that the model is overfitting the training data set.

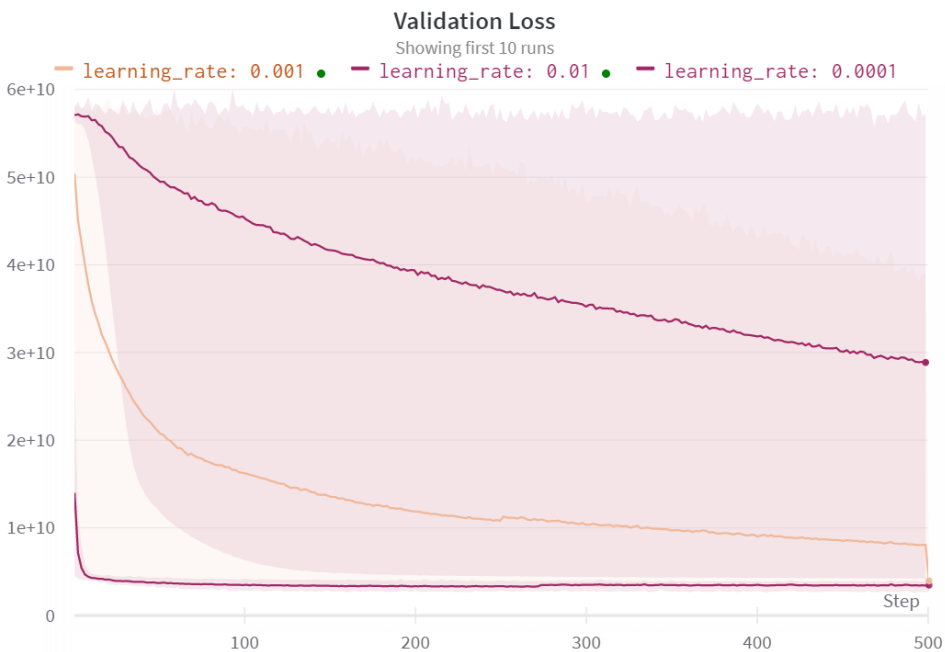
Rank	batch_size	learning_rate	num_epochs	shape	Test Loss	Training Loss	Rel diff
1	64	0.01	1000	9-128-128-1	51.769	45.215	0.29
2	16	0.001	1000	9-256-256-1	52.109	49.666	0.05
3	32	0.01	1000	9-128-128-1	53.495	46.561	0.15
4	16	0.001	1000	9-128-128-1	53.809	52.234	0.03
5	16	0.01	1000	9-128-128-1	53.928	40.689	0.33
6	32	0.001	1000	9-128-128-1	55.738	54.268	0.03
7	32	0.001	500	9-128-128-1	57.919	57.489	0.01
8	32	0.001	500	9-64-64-1	59.143	58.992	0.00
9	16	0.01	1000	9-256-1	60.501	60.264	0.00
10	512	0.001	500	9-128-128-1	61.361	64.091	-0.04
11	32	0.01	1000	9-256-1	62.263	62.087	0.00
12	128	0.001	500	9-128-128-1	62.376	62.483	-0.00
13	32	0.01	1000	9-128-1	62.596	62.165	0.01
14	128	0.001	500	9-64-64-1	62.709	62.579	0.00
15	16	0.01	1000	9-256-256-1	62.767	25.870	1.43
16	16	0.01	1000	9-128-1	62.813	62.063	0.01
17	16	0.001	1000	9-256-1	63.699	63.860	-0.00
18	32	0.001	1000	9-256-1	64.188	64.479	-0.00
19	512	0.001	500	9-64-64-1	64.529	65.238	-0.01
20	32	0.0001	500	9-128-128-1	64.990	65.455	-0.01
21	32	0.001	1000	9-128-1	65.020	65.183	-0.00
22	32	0.0001	500	9-64-64-1	66.326	66.945	-0.01
23	16	0.001	1000	9-128-1	66.353	64.580	0.03
24	32	0.001	500	9-128-1	66.406	66.991	-0.01
25	32	0.001	500	9-64-1	67.004	67.758	-0.01
26	128	0.0001	500	9-128-128-1	68.877	69.421	-0.01
27	128	0.001	500	9-128-1	72.188	73.397	-0.02
28	128	0.0001	500	9-64-64-1	81.885	82.803	-0.01
29	128	0.001	500	9-64-1	88.378	89.210	-0.01
30	32	0.0001	500	9-128-1	129.872	131.370	-0.01
31	512	0.0001	500	9-128-128-1	134.686	138.144	-0.03
32	512	0.001	500	9-128-1	160.913	160.687	0.00
33	32	0.0001	500	9-64-1	172.261	172.242	0.00
34	512	0.001	500	9-64-1	196.424	195.188	0.01
35	512	0.0001	500	9-64-64-1	200.460	200.965	-0.00
36	128	0.0001	500	9-128-1	227.688	225.577	0.01
37	128	0.0001	500	9-64-1	233.420	232.149	0.01
38	512	0.0001	500	9-128-1	240.643	235.966	0.02
39	512	0.0001	500	9-64-1	242.282	236.436	0.02

3.2 Batch Size



We tested with batch sizes of 16, 32, 128, and 512. We wanted to test with smaller and larger batch sizes. Smaller batch sizes minimized loss much faster than larger ones, which ultimately resulted in a lower loss for those configurations that had lower batch size due to our low number of epochs. From external reading, higher batch sizes perform better due to having a better estimate of the gradient however, due to potentially having a lower number of epochs among other attributes, our results do not reflect this.

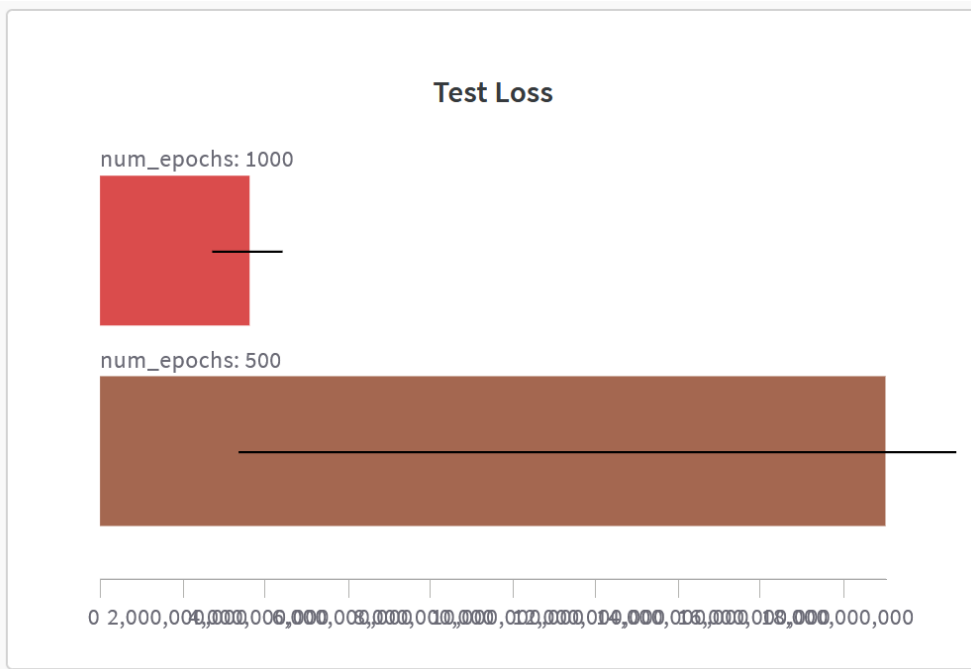
3.3 Learning Rate



In our tests, we observed that higher initial learning rates resulted in a faster minimization of loss.

Although using a lower learning rate would have resulted in less volatile test and validation loss statistics, we chose to use learning rates like 0.01 and 0.001 in order to save time spent on training the network.

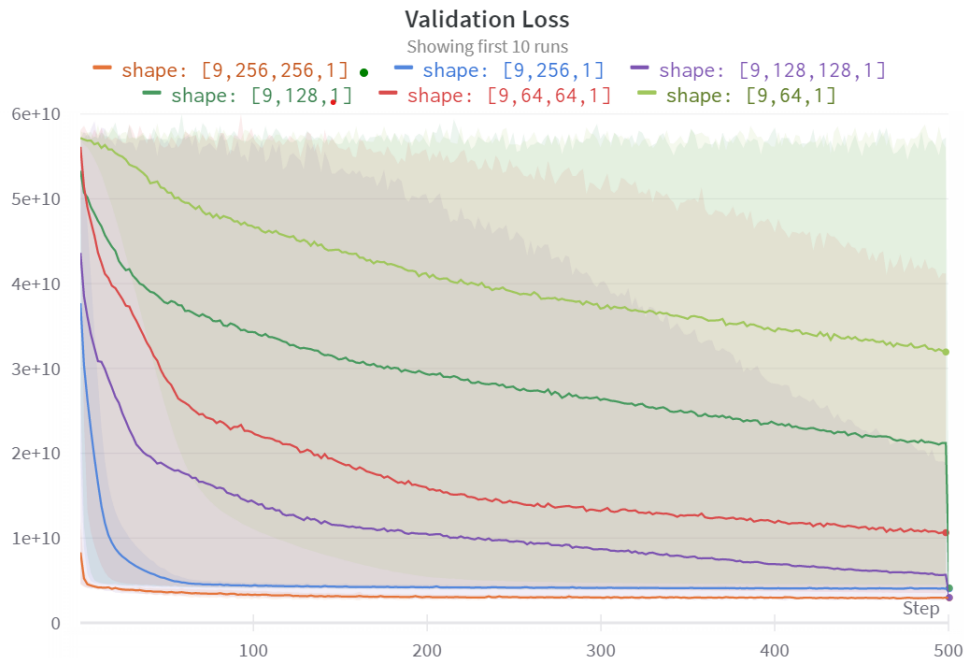
3.4 Number of Epochs



We decided to test either 500 or 1000 epochs. We observed that, in general, models that were trained over 1,000 epochs produced a lower test loss at the end - implying that it improved the overall performance of the model without resulting in overfitting.

We could likely have used a greater number of epochs in order to determine at which point exactly overfitting begins to occur.

3.5 Neural Network Shape



In our experiments, we observed that the use of networks with more neurons within their hidden layers reached lower test loss than those with less.

However, among those networks with a given number of neurons contained within their hidden layers, those with higher depth performed better than their lower depth counterparts - i.e. 256 neurons spread across two layers of 128 had a lower test loss than a single layer of 256 neurons. This means more hidden layers perform better than increasing number of nodes in a layer. Multiple layers mean more features can be learned and implies

4 Evaluation of Best Model

The configuration with the lowest loss on the test data set turned out to be the one with batch size: 16, learning rate: 0.001, number of epochs: 1000, and shape 9-256-256-1 (layers: 9x256 ReLU, 256x256 ReLU, 256x1 linear). It had a MSE of 51,769.

This is, surprisingly, inconsistent from some of our observations about the effect of various hyperparameters on test loss. In spite of being the best-performing neural network, it has only 128 neurons in its 2 hidden layers (although it is consistent with the finding that higher depth results in better performance) and uses a batch size of 64, rather than 16.

It's possible that the lower batch size and higher neuron counts of some of its nearby competitors resulted in underfitting.